Web-Technologien UE - Assignment 7

Raphael Bleier, WS 2026

Gleiches Setup wie bei Blatt 6 (Node 22, PostgreSQL 18). Getestet mit curl, in ex1 und ex2 liegt jeweils eine `postman_collection.json`. Vor dem Start `npm install`.

Exercise 1 - JWT Authentication

Die Datenbank `webtech_jwt` hab ich mit dem mitgelieferten Skript importiert, die geht ohne Probleme, weil der Name schon klein geschrieben ist. pool.js und die Produkt-Routen sind von 6.4 übernommen. Session, Logout und express-session sind weg, weil JWT ohne Server-State auskommt.

login.js: Gleiche DB-Abfrage wie in 6.4 (parametrisiert). Bei Erfolg wird der Token mit `jwt.sign({ login }, "secret", { expiresIn: "1h" })` erzeugt, key und Ablaufzeit kommen aus der config.json. Im Payload steht der Login-Name, damit man später weiß wer den Request schickt. Antwort: message, login und token. Falsche Daten geben 401 "login failed".

check_auth.js: Alles steht in einem try-catch. Der Token kommt aus `req.headers.authorization`, ein "Bearer " davor wird abgeschnitten (Postman schickt das so). `jwt.verify(token, "secret")` prüft Signatur und Ablaufzeit und gibt den Payload zurück, der landet in `req.userData`. Danach `next()`. Ist der Token falsch, abgelaufen oder fehlt der Header ganz, wirft irgendwas im try einen Fehler und der catch gibt 401 zurück.

Unterschied zur Session (6.4): Der Server speichert nichts. Der ganze Login-Zustand steckt im Token selbst und ist mit dem Secret signiert, deshalb kann ihn niemand verändern ohne dass die Signatur kaputt geht. Lesen kann ihn aber jeder, der Payload ist nur Base64. Also keine Passwörter oder anderen Geheimnisse reinschreiben. Nachteil: Ausloggen geht nicht wirklich, ein Token ist gültig bis er abläuft.

```
$ curl -i http://localhost:3000/products
HTTP/1.1 401 Unauthorized
Content-Length: 35

{"message":"Authentication failed"}

$ curl -i -X POST -H 'Content-Type: application/json' -d '{"user":"admin","pass":"falsch"}' http://local
host:3000/login
HTTP/1.1 401 Unauthorized
Content-Length: 26

{"message":"login failed"}

$ curl -i -X POST -H 'Content-Type: application/json' -d '{"user":"admin","pass":"admin"}' http://localh
ost:3000/login
HTTP/1.1 200 OK
Content-Length: 206

{"message":"login successful","login":"admin","token":"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJsb2dpbiI6
ImFkbWluIiwiaWF0IjoxNzkxMzY5MDI5LCJleHAiOjE3OTEzNzI2Mjl9.WPeCcqWniqc_u6jpRO4MSveQ3S7CQ5nuRny8eOM-yss"}

$ TOKEN=<token von oben>
```

```
$ curl -i -H "Authorization: $TOKEN" http://localhost:3000/products
HTTP/1.1 200 OK
Content-Length: 66

{"productIds":["NiconDX16400","sony24105","sonyAIV","tamron1724"]}

$ curl -i -H "Authorization: Bearer $TOKEN" http://localhost:3000/product/sonyAIV
HTTP/1.1 200 OK
Content-Length: 96

{"id":"sonyAIV","title":"Sony Alpha IV","description":"New 2021 arrival, hot deal","price":2890}

$ curl -i -H "Authorization: ${TOKEN}kaputt" http://localhost:3000/products
HTTP/1.1 401 Unauthorized
Content-Length: 35

{"message":"Authentication failed"}

# token payload (mitte vom token, base64 dekodiert):
{"login":"admin","iat":1791369029,"exp":1791372629}
```

Exercise 2 - Product List REST Service

DB `webtech_rest` importiert. Neu sind `likes_count`/`dislikes_count` in products sowie die Tabellen `tags` und `products_tags`. Letztere ist die n:m Zuordnung, weil ein Produkt mehrere Tags haben kann und ein Tag bei mehreren Produkten vorkommt.

Routen

Methode	URL	Was	Status
GET	`/products`	Liste, mit Filtern (siehe unten)	200, 400
GET	`/products/:id`	Ein Produkt mit Tags und Rating	200, 404
POST	`/products`	Anlegen, Body: id, title, description, price	201 + Location, 400, 409
PUT	`/products/:id`	Titel, Beschreibung, Preis ersetzen	200, 400, 404
DELETE	`/products/:id`	Löschen inkl. Tag-Zuordnungen	204, 404
POST	`/products/:id/likes`	Like +1	200, 404
POST	`/products/:id/dislikes`	Dislike +1	200, 404
GET	`/products/:id/tags`	Tags vom Produkt	200, 404
POST	`/products/:id/tags`	Tag dazu, Body: {"tag": "..."}	201, 400, 404, 409
DELETE	`/products/:id/tags/:tag`	Tag entfernen	204, 404

Warum so: Die Ressource ist das Produkt, alles was dazugehört (Likes, Tags) ist eine Unterressource darunter. Die HTTP-Methode sagt was passiert, deshalb gibt es keine Verben in den URLs wie `/likeProduct`. Likes sind POST und nicht PUT, weil jeder Aufruf den Zähler erhöht, das ist nicht idempotent.

Suche

Alles über Query-Parameter an `/products`, die man auch kombinieren kann:

`?search=text` sucht den Teilstring (ohne Groß/Kleinschreibung, ILIKE) in Titel, Beschreibung oder einem der Tags.
`?minPrice=500&maxPrice=1000` ist das Preisintervall, Länge frei wählbar, z.B. 500 oder 1000.
`?minRating=0.6&maxRating=0.8` ist das Rating-Intervall mit Rating = L/(L+D). In 1/5-Schritten entspricht das den Sternen, 0.8 bis 1 sind also 5 Sterne. Produkte ohne Bewertung (L+D = 0) haben rating null und fallen bei der Rating-Suche raus, sonst würde man durch 0 teilen.

Probleme beim Umsetzen

Die Tabellen haben keinen Primary Key. Deshalb ging `GROUP BY p.id` nicht, Postgres verlangt dann alle Spalten im GROUP BY. Doppelte IDs verhindert auch nichts, deshalb prüft der POST zuerst selbst, ob es die ID schon gibt (dann 409). Beim Einfügen eines neuen Tags musste ich `$1::varchar` casten, weil Postgres den Typ vom Parameter in `SELECT $1 WHERE NOT EXISTS ...` nicht erkennt.

```
$ curl -i -X GET "http://localhost:3000/products/sony24105"
HTTP/1.1 200 OK

{"id":"sony24105","title":"Sony 24-105mm lens","description":"Best Sony travel zoom","price":1300,"likes
_count":3,"dislikes_count":8,"tags":["sony","travel","zoom"],"rating":0.2727272727272727}

$ curl -i -X GET "http://localhost:3000/products?search=travel"
HTTP/1.1 200 OK

[{"id":"NiconDX16400","title":"Nicon DX 16-600","description":"Nicon crop travel lens","price":900,"like
s_count":12,"dislikes_count":1,"tags":["lens","nicon","travel"],"rating":0.9230769230769231},{"id":"sony
24105","title":"Sony 24-105mm lens","description":"Best Sony travel zoom","price":1300,"likes_count":
3,"dislikes_count":8,"tags":["sony","travel","zoom"],"rating":0.2727272727272727}]

$ curl -i -X GET "http://localhost:3000/products?search=Alpha"
HTTP/1.1 200 OK

[{"id":"sonyAIV","title":"Sony Alpha IV","description":"New 2021 arrival, hot deal","price":2890,"likes_
count":5,"dislikes_count":2,"tags":["alpha","sony"],"rating":0.7142857142857143}]

$ curl -i -X GET "http://localhost:3000/products?minPrice=500&maxPrice=1000"
HTTP/1.1 200 OK

[{"id":"NiconDX16400","title":"Nicon DX 16-600","description":"Nicon crop travel lens","price":900,"like
s_count":12,"dislikes_count":1,"tags":["lens","nicon","travel"],"rating":0.9230769230769231},{"id":"tamr
on1724","title":"Tamron 17-24mm","description":"Full frame 17-14mm 2.8 lens","price":800,"likes_count":
0,"dislikes_count":0,"tags":["lens","tamron"],"rating":null}]

$ curl -i -X GET "http://localhost:3000/products?minRating=0.8&maxRating=1"
HTTP/1.1 200 OK

[{"id":"NiconDX16400","title":"Nicon DX 16-600","description":"Nicon crop travel lens","price":900,"like
s_count":12,"dislikes_count":1,"tags":["lens","nicon","travel"],"rating":0.9230769230769231}]

$ curl -i -X GET "http://localhost:3000/products?minPrice=abc"
HTTP/1.1 400 Bad Request

{"message":"minPrice must be a number"}

$ curl -i -X POST -H 'Content-Type: application/json' -d '{"id":"canonR6","title":"Canon EOS R6","descri
ption":"Full frame mirrorless","price":2100}' "http://localhost:3000/products"
HTTP/1.1 201 Created
Location: /products/canonR6
```

```
{"id":"canonR6","title":"Canon EOS R6","description":"Full frame mirrorless","price":2100,"likes_count":
0,"dislikes_count":0,"tags":[],"rating":null}

$ curl -i -X POST -H 'Content-Type: application/json' -d '{"id":"canonR6","title":"Canon EOS R6","descri
ption":"Full frame mirrorless","price":2100}' "http://localhost:3000/products"
HTTP/1.1 409 Conflict

{"message":"Product canonR6 already exists"}

$ curl -i -X POST -H 'Content-Type: application/json' -d '{"id":"x","title":"ohne preis"}' "http://local
host:3000/products"
HTTP/1.1 400 Bad Request

{"message":"id, title, description and price (integer) are required"}

$ curl -i -X POST "http://localhost:3000/products/canonR6/likes"
HTTP/1.1 200 OK

{"id":"canonR6","title":"Canon EOS R6","description":"Full frame mirrorless","price":2100,"likes_count":
1,"dislikes_count":0,"tags":[],"rating":1}

$ curl -i -X POST "http://localhost:3000/products/canonR6/likes"
HTTP/1.1 200 OK

{"id":"canonR6","title":"Canon EOS R6","description":"Full frame mirrorless","price":2100,"likes_count":
2,"dislikes_count":0,"tags":[],"rating":1}

$ curl -i -X POST "http://localhost:3000/products/canonR6/dislikes"
HTTP/1.1 200 OK

{"id":"canonR6","title":"Canon EOS R6","description":"Full frame mirrorless","price":2100,"likes_count":
2,"dislikes_count":1,"tags":[],"rating":0.6666666666666666}

$ curl -i -X POST -H 'Content-Type: application/json' -d '{"tag":"canon"}' "http://localhost:3000/produc
ts/canonR6/tags"
HTTP/1.1 201 Created

["canon"]

$ curl -i -X POST -H 'Content-Type: application/json' -d '{"tag":"camera"}' "http://localhost:3000/produ
cts/canonR6/tags"
HTTP/1.1 201 Created

["camera","canon"]

$ curl -i -X POST -H 'Content-Type: application/json' -d '{"tag":"canon"}' "http://localhost:3000/produc
ts/canonR6/tags"
HTTP/1.1 409 Conflict

{"message":"Product already has tag canon"}

$ curl -i -X GET "http://localhost:3000/products/canonR6/tags"
HTTP/1.1 200 OK

["camera","canon"]

$ curl -i -X GET "http://localhost:3000/products?search=canon&minRating=0.6&maxRating=0.8"
HTTP/1.1 200 OK
```

```
[{"id":"canonR6","title":"Canon EOS R6","description":"Full frame mirrorless","price":2100,"likes_coun
t":2,"dislikes_count":1,"tags":["camera","canon"],"rating":0.6666666666666666}]

$ curl -i -X DELETE "http://localhost:3000/products/canonR6/tags/camera"
HTTP/1.1 204 No Content

$ curl -i -X DELETE "http://localhost:3000/products/canonR6/tags/camera"
HTTP/1.1 404 Not Found

{"message":"Product canonR6 has no tag camera"}

$ curl -i -X PUT -H 'Content-Type: application/json' -d '{"title":"Canon EOS R6 II","description":"Full
frame mirrorless, 2nd gen","price":2400}' "http://localhost:3000/products/canonR6"
HTTP/1.1 200 OK

{"id":"canonR6","title":"Canon EOS R6 II","description":"Full frame mirrorless, 2nd gen","price":2400,"l
ikes_count":2,"dislikes_count":1,"tags":["canon"],"rating":0.6666666666666666}

$ curl -i -X DELETE "http://localhost:3000/products/canonR6"
HTTP/1.1 204 No Content

$ curl -i -X GET "http://localhost:3000/products/canonR6"
HTTP/1.1 404 Not Found

{"message":"Product canonR6 not found"}

$ curl -i -X POST "http://localhost:3000/products/gibtsnicht/likes"
HTTP/1.1 404 Not Found

{"message":"Product gibtsnicht not found"}
```

Exercise 3 - REST Spezifikation Bibliothek

Die komplette Spezifikation liegt als OpenAPI 3 in `ex3/openapi.yaml`. Die kann man z.B. in editor.swagger.io einfügen und sich ansehen. Hier die Zusammenfassung.

Ressourcen

`/books` ist die Sammlung aller Bücher. `/books/{isbn}` ist ein einzelnes Buch, die ISBN ist der Schlüssel. `/books/{isbn}/rental` ist die Ausleihe von einem Buch. Die hab ich als eigene Unterressource gemacht und nicht als Aktion wie `/books/{isbn}/rent`: Ausleihen heißt die Ausleihe anlegen (PUT), Zurückgeben heißt sie löschen (DELETE), und den Status fragt man mit GET ab. So reichen die normalen HTTP-Methoden und man braucht keine Verben in der URL.

Funktion	Methode	URL	Body	Antwort
`getAllBooks()`	GET	`/books`	-	200 Liste
`getBook(isbn)`	GET	`/books/{isbn}`	-	200 Buch, 404
`isRented(isbn)`	GET	`/books/{isbn}/rental`	-	200 {rented: true/false}, 404
`searchByTitle(titleString)`	GET	`/books?title={titleString}`	-	200 Liste (evtl. leer)

`searchByAuthor(authorName)`	GET	`/books?author={authorName}`	-	200 Liste
`searchByAttribute(attr, str)`	GET	`/books?searchIn={attr}&q={str}`	-	200 Liste, 400 bei unbekanntem Attribut
`rent(isbn)`	PUT	`/books/{isbn}/rental`	-	200, 404, 409 schon ausgeliehen
`return(isbn)`	DELETE	`/books/{isbn}/rental`	-	204, 404, 409 nicht ausgeliehen
`createBook(book)`	POST	`/books`	Book als JSON	201 + Location, 400, 409 ISBN existiert
`updateBook(isbn, book)`	PUT	`/books/{isbn}`	Book als JSON	200, 400, 404
`deleteBook(isbn)`	DELETE	`/books/{isbn}`	-	204, 404, 409 ist ausgeliehen

Book = die Struktur aus book_library.json: isbn, title, pageCount, publishedDate, thumbnailUrl, shortDescription, longDescription, status, authors[], categories[], rented (nur lesen, wird über /rental geändert).

Entscheidungen

Suchen sind Query-Parameter auf /books und keine eigenen Ressourcen. Das Ergebnis ist ja immer noch eine Liste von Büchern, nur gefiltert. title sucht einen Teilstring, author vergleicht laut Angabe auf Gleichheit mit einem der Autoren. Für searchByAttribute gibt es `searchIn` (title, shortDescription, longDescription, author, category) plus `q` für den Suchbegriff.

rent ist PUT, weil es idempotent ist. Ein Buch zweimal auszuleihen ändert nichts am Zustand, es ist einfach ausgeliehen. Beim zweiten Mal kommt 409, damit der Client merkt, dass schon jemand anderer das Buch hat. POST ginge auch, aber pro Buch gibt es immer nur genau eine Ausleihe.

updateBook ist PUT und ersetzt das ganze Buch. Für einzelne Felder könnte man zusätzlich PATCH anbieten.

Statuscodes: 200/201/204 für Erfolg, 400 für kaputte Eingaben, 404 wenn es die ISBN nicht gibt, 409 für Konflikte mit dem Zustand (ISBN existiert schon, schon ausgeliehen, nicht ausgeliehen, löschen während ausgeliehen).

Datenproblem: In book_library.json haben 2 Bücher gar keine ISBN und ein paar ISBNs kommen doppelt vor (z.B. 1884777686). Weil die ISBN hier der Schlüssel in der URL ist, müsste man das vor dem Import bereinigen, sonst wären die Bücher nicht eindeutig erreichbar.